



Secure

Unlock an Android phone by bruteforcing the lockscreen PIN

Feb 2

OTG cable
Male Micro-
Female USB-

Android

Unlock the lockscreen for the

Turn on the Nethunter for Android devices!

? How it works

It uses a USB OTG cable to connect the locked phone to the Nethunter device. It emulates a keyboard, automatically tries PINs, and waits after trying too many wrong guesses

Go from  Kickoff 11:30AM to winning the pitch

Jessica



Search For Owner Full Name, Full Address

Example: (123) 456-7890

() -

The USB HID Gadget driver provides emulation of USB Human Interface Devices (HID). This enables an Android Nethunter device to emulate keyboard input to the locked phone. It's just like plugging a keyboard into the locked phone and pressing keys.

⌚ This takes a bit over 16.6 hours to try all possible 4 digit PINs, but with the optimised PIN list it should take you much less time.

? 🛠 ? You will need

- A locked Android phone
- A Nethunter phone (or any rooted Android with HID kernel support)
- USB OTG (On The Go) cable/adaptor (USB male Micro-B to female USB A), and a standard charging cable (USB male Micro-B to male A).
- That's all!

? Benefits

- Turn your NetHunter phone into an Android PIN cracking machine
- Unlike other methods, you do not need ADB or USB debugging enabled on the locked phone
- The locked Android phone does not need to be



- It works!

★ Features

- Optimised PIN list
- Bypasses phone pop-ups including the Low Power warning
- Detects when the phone is unplugged or powered off, and waits while retrying every 5 seconds
- Configurable delays of N seconds after every X PIN attempts
- Log file

Installation

TBC

Executing the script

If you installed the script to /sdcard/, you can execute it with the following command.

```
bash ./android-pin-bruteforce
```

Note that Android mounts /sdcard with the noexec flag. You can verify this with `mount .`

Usage

Android-PIN-Bruteforce (0.1) is used to unlock
Find more information at: <https://github.com/>

Commands:

crack	Begin cracking PINs
resume	Resume from a chosen PIN



Android Example 365

```
-t, --type TYPE    Select PIN or PATTERN crack
-l, --length NUM   Crack PINs of NUM length
-d, --dry-run      Dry run for testing. Doesn'
```

Usage:

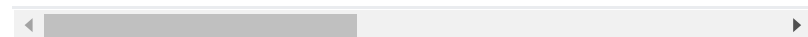
```
android-pin-bruteforce <command> [options]
```

Supported Android Phones/Devices

It has been tested with these devices:

- Samsung S5 with Android 6.0.1

? PIN Lists



Optimised PIN list

pinlist.txt is an optimised list of all possible 4 digit PINs, sorted by order of likelihood.

pinlist.txt is from

<https://github.com/mandatoryprogrammer/droidbrute>

This list is used with permission from Justin Engler & Paul Vines from Senior Security Engineer, iSEC Partners,

and was used in their Defcon talk, [Electromechanical PIN Cracking with Robotic Reconfigurable Button Basher \(and C3RO\)](#)

Cracking with Masks

Masks use regular expressions with the standard grep extended format.

```
./android-pin-bruteforce crack --mask "...[45]"
--dry-run
```



- To try PINs that end in 4 or 5, use ... [45]

Different Lock Screens

Device manufacturers create their own lock screens that are different to the default or stock Android.

You can customise the timing and prompt keys to bring up the unlock screen by setting variables in the config file.

Default configuration for Android 6.0

This is the default configuration. It sends 5 PINs before waiting for a cooldown timeout of 30 seconds. Before each PIN is sent it sends a prompt of the ESCAPE and ENTER keys to bring up the unlock screen.

```
DELAY_BETWEEN_KEYS=0.1
COOLDOWN_TIME=30
COOLDOWN_AFTER_N_ATTEMPTS=5
CHANGE_AFTER_10_ATTEMPTS=0
PROMPT_BEFORE_EACH_PIN="escape enter"
```

Configuration for Android 10

This configuration sends a lockscreen prompt of escape and space before each PIN is sent.

It has a 30 second cooldown after each attempt

After 10 attempts, the cooldown will occur after each PIN attempt.

```
DELAY_BETWEEN_KEYS=0.1
COOLDOWN_TIME=30
COOLDOWN_AFTER_N_ATTEMPTS=5
CHANGE_AFTER_10_ATTEMPTS=1
PROMPT_BEFORE_EACH_PIN="escape space"
```





- [DONE] Improve Usage and commandline options/config files
- [DONE] Add bruteforce for n digit PINs
- [DONE] Mask for known digits
- [DONE] Crack PIN list in reverse (to find which recent PIN unlocked the device)
- [DONE] Implement configurable lockscreen prompt
- [DONE] Implement cooldown change after 10 attempts
- Add progress bar
- Add ETA
- ASCII art
- Nicer GUI for NetHunter
- Find/test more devices to bruteforce (iPhone)
- Try to detect when phone is unlocked (Use Nethunter camera as a sensor?)
- Crack Android Patterns (try common patterns first)

? Troubleshooting

If it is not bruteforcing PINs

Check the orientation of the cables

The Nethunter phone should have a regular USB cable attached, while the locked phone should have an OTG adaptor attached.

The OTG cable should be connected to the locked Android phone. The regular USB cable should be connected to the Nethunter phone.

Refer to the graphic on how to connect the phones.





Android Example 365

using a regular charging/data USB cable. Open a text editor like Notepad while it is cracking and you should see it entering PIN numbers into the text editor.

Note that you will not need an OTG cable for this.

Try restarting the phones

Try powering off the phones and even taking out the batteries if that is possible.

Try new cables

Try using new cables/adaptors as you may have a faulty cable/adaptor.

If it doesn't unlock the phone with a correct PIN

You might be sending keys too fast for the phone to process. Increase the DELAY_BETWEEN_KEYS variable in the config file.

? If you don't see 4 dots come up on the phone's screen then maybe it is not receiving 4 keys.

? If your phone runs out of power too soon

Try using a USB OTG cable that has an external power supply. This will charge the phone's battery while it operates.

Check the Diagnostics Report

Use the command `diag display` diagnostic information.

```
bash ./android-pin-bruteforce diag
```

If you receive this message when the USB cable is plugged in then try taking the battery out of the locked Android phone and power cycling it.





as expected. This is especially useful when you are modifying the configuration.

Other

Try this command in a shell on the NetHunter phone:

```
/system/bin/setprop sys.usb.config hid
```

? Known Issues

- This cannot detect when the correct PIN is guessed and the phone unlocks.
- Your phones may run out of ? battery before the correct PIN is found.

? Technical Details

This works from an Android phone because the USB ports are not bidirectional, unlike the ports on a laptop.

Keys are sent using `/system/xbin/hid-keyboard`. To test this and send the key 1 you can use `echo 1 | /system/xbin/hid-keyboard dev/hidg0 keyboard`



InfluxDB Cloud

Run and grow large
time series data
workloads at high
volumes globally

Try For Free

Before each PIN, we send the escape and enter keys.



Android Example 365

program for testing the HID gadget driver that is included in the Linux Kernel. The source code for this file can be found at https://www.kernel.org/doc/html/latest/usb/gadget_hid.html and https://github.com/aagallag/hid_gadget_test.

The diagnostics command uses the `usb-devices` script but it is only necessary as part of determining whether the USB cables are incorrectly connected. This can be downloaded from <https://github.com/gregkh/usbutils/blob/master/usb-devices>

How to send special keys

To send special keys use the following labels. This list is from the `hid_gadget_test` source code.

- left-ctrl
- right-ctrl
- left-shift
- right-shift
- left-alt
- right-alt
- left-meta
- right-meta
- return
-  
- bckspc 
- tab
- spacebar
- caps-lock
- f1
- f2



- f7
- f8
- f9
- f10
- f11
- f12
- insert
- home
- pageup
- del
- end
- pagedown
- right
- left
- down
- kp-enter
- up
- num-lock

To send combinations of keys use the following list:

- ctrl_escape

If you need more key combinations please open a new issue in the [GitHub issues list](#).

? Contributing

Pull requests are welcome. For major changes, please open an issue first to discuss what you would like to change.



Motivation

My original motivation to develop this was to unlock a Samsung S5 Android phone. It had belonged to someone who had passed away, and their family needed access to the data on it. As I didn't have a USB Rubber Ducky or any other hardware handy, I tried using a variety of methods, and eventually realised I had to develop something new.

Credit

The optimised PIN list is from Justin Engler (@justinengler) & Paul Vines from Senior Security Engineer, iSEC Partners and was used in their Defcon talk, [Electromechanical PIN Cracking with Robotic Reconfigurable Button Basher \(and C3BO\)](#)..

Graphics

Designed by Andrew Horton and gratefully using these free vector packs:

- [USB Ports Isometric Free Vector by VisionHeldup](#)
- [HDMI and USB Vector Set by Mary Winkler](#)
- [Isometric Data Security Illustration by Rizal.Medanguide](#)
- Kali NetHunter Logo

? Comparison with other projects and methods to unlock a locked Android phone

What makes this project unique?

I've been asked what makes this project unique when





Android Example 365

doesn't need the locked phone to be pre-hacked.

It works:

- Without having to buy special hardware, such as a Rubber Ducky, Celebrite, or XKEY.
- Without ADB or root access (the phone doesn't have to be pre-hacked).

Project	ADB/ USB Debu gging	Req uire s root	Requir es \$ hardw are	Co mm erci al
★ Android-PIN-Bruteforce	No	No	Nethu nter phone	No
github.com/PentesterES/AndroidPINCrack	Yes	Yes	No	No
github.com/ByteRockstar1996/Cracking-Android-Pin-Lock	Yes	Yes	No	No
github.com/sch3m4/androidpatternlock	Yes	Yes	No	No
github.com/georgenicolaou/androidlockcracker	Yes	Yes	^ No	No
github.com/MGF15/P-Decode	Yes	Yes	No	No
github.com/Bite	Yes	Yes	No	No



Android Example 365

	Debu gging	s root	hardw are	erci al
TER				
github.com/Gh 005t/Android- BruteForce	Yes	No	No	No
github.com/ma ndatoryprogra mmer/droidbrut e	No	No	Rubbe r Ducky \$	No
github.com/hak 5darren/USB- Rubber-Ducky	No	No	Rubbe r Ducky \$	Yes
github.com/bbr other/stm32f4a ndroidbruteforc e	No	No	STM3 2F4 dev board \$	No
hdb- team.com/prod uct/hdbox/	No	No	HDBO X \$\$	Yes
xpinclip.com	No	No	XPIN Clip \$\$	Yes
^ cellebrite.com/ en/ufed/	No	No	X Celleb rite UFED \$\$\$	Yes

Some of these projects/products are really awesome but they achieve a different goal to Android-PIN-Bruteforce.



If a project requires a gestures key or password key I've



There are links to each of these projects in the :
Related Projects & Futher Reading section.

? Regular phone users

- Try the top 20 PINs from the [DataGenetics PIN analysis](#) that apparently unlocks 26.83% of phones.
- Use an SMS lock-screen bypass app (requires app install before phone is locked)
- Use Samsung Find My Mobile (requires you set it up before phone is locked)
- Crash the Lock Screen UI (Android 5.0 and 5.1)
- Use the Google Forgot pattern, Forgot PIN, or Forgot password (Android 4.4 KitKat and earlier)
- Factory Reset (you lose all your data ?)

? Users who have already replaced their Android ROM

If the phone has already been rooted, has USB debugging enabled, or has adb enabled.

- Flash the Pattern Password Disable ZIP using a custom recovery (Requires TWRP, CMW, Xrec, etc.)
- Delete /data/system/gesture.key or password.key (requires root and adb on locked device)
- Crack /data/system/gesture.key and password.key (requires root and adb on locked device)
- Update sqlite3 database settings.db (requires root and adb on locked device)

? Forensic Investigators





Android Example 365

- Bruteforce with keyboard emulation (★ Android-PIN-Bruteforce, RubberDucky attack, XPIN Clip, HBbox)
- JTAG (Interface with TAPs (Test Access Ports) on the device board)
- In-System Programming (ISP) (Involves directly connecting to pins on flash memory chips on the device board)
- Chip Off (Desolder and remove flash memory chips from the device)
- Clock Glitching / Voltage Fault Injection (Hardware CPU timing attacks to bypass PIN restrictions)
- Bootloader exploits (Zero-day exploits that attack the bootloader. GrayKey from Grayshift and Cellebrite)

JTAG, ISP, and Chip Off techniques are less useful now because most devices are encrypted.

I don't know of any practical attacks on phone PINs that use clock glitching, if you know of a product that uses this technique please let me know so I can include it.

? Security Professionals and Technical Phone Users

Use the USB HID Keyboard Bruteforce with some dedicated hardware.

- ~~A RubberDucky and Darren Kitchen's Hak5 bruteforce script~~
- Write a script for a USB Teensy
- Buy expensive forensic hardware
- Or you can use Android-PIN-Bruteforce with your NetHunter phone!



Attempts to use an otherwise awesome project Duck



No Comments Yet

Type Comment Here (at least 3 chars)

Name (opti

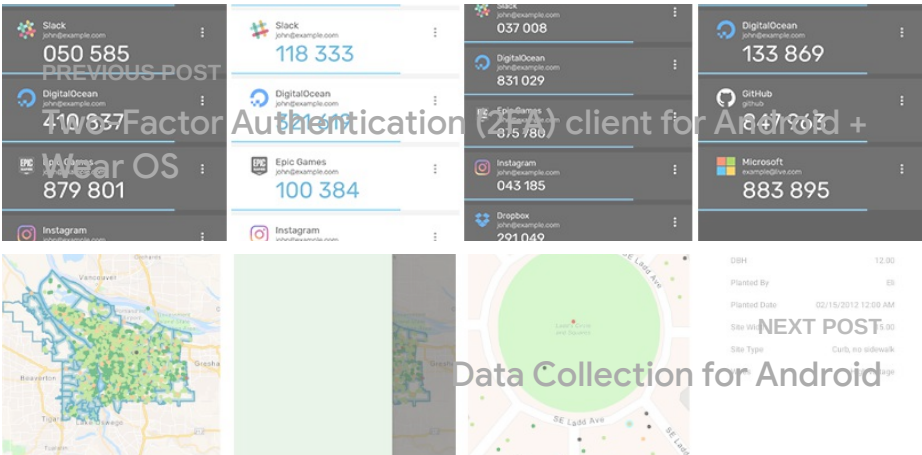
E-mail (opti

Website (o

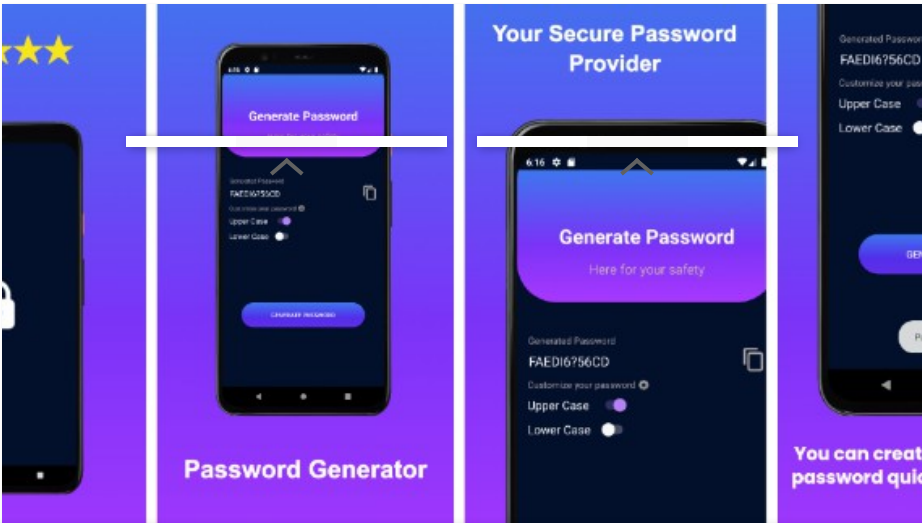
Submit

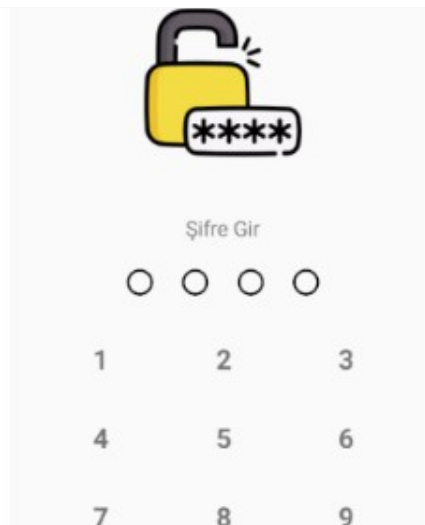
Preview

#Secure #Password



YOU MIGHT ALSO LIKE...





Password

You can store all your password, bank details, card details in one place and remember only one master PIN

You can store all your password, bank details, card details in one place and remember only one master PIN

13 JANUARY 2022



Enter Password



Ok



Easy



Medium



Strong



VeryStrong

Password

PasswordStrengthView: an android



Subscribe to Android Example 365

Get the latest posts delivered right to your inbox

youremail@example.com

Subscribe

[About](#) | [Contact](#) | [Privacy Policy](#) | [Submit A Project](#)

Powered by [Android Example 365](#)

